SBA301 – LAB 05

CREATE SINGLE PAGE APPLICATION WITH SPRING BOOT RESTFUL API

HƯỚNG DẪN THỰC HÀNH CHI TIẾT STEP BY STEP

ReactJS / Vite ↔ Axios ↔ Spring Boot 3 ↔ Spring Data JPA ↔ SQL Server

Case study: Orchid Management

Course baseline: JDK 21 • Spring Boot 3.x • Maven • ReactJS • Vite • Axios • MS SQL Server

Mục tiêu tài liệu: Sinh viên có thể bắt đầu từ môi trường mới, cài đặt và cấu hình đầy đủ FE/BE/DB, triển khai CRUD, tích hợp CORS/API URL, chạy hệ thống và tự kiểm chứng từng lớp bằng Postman, Browser DevTools và SQL Server.

1. TÓM TẮT NỘI DUNG VÀ YÊU CẦU

Lab 05 yêu cầu xây dựng một ứng dụng full-stack cho Orchid Management. Backend cung cấp RESTful API bằng Spring Boot 3 và lưu dữ liệu bằng Spring Data JPA trên SQL Server; frontend ReactJS gửi HTTP request bằng Axios và hiển thị dữ liệu trên các component. Tài liệu này giữ đúng mục tiêu đó và mở rộng thành quy trình triển khai có checkpoint sau từng bước.

Hạng mục	Yêu cầu đầu ra
Backend	Spring Boot REST API hỗ trợ GET list/detail, POST, PUT, DELETE cho Orchid.
Persistence	SQL Server lưu dữ liệu Orchid; JPA repository thực hiện CRUD.
Architecture	Controller → Service → Repository → SQL Server; frontend không truy cập DB trực tiếp.
Frontend	React/Vite hiển thị danh sách, điều hướng, tạo/cập nhật/xóa dữ liệu qua Axios.
Integration	API base URL rõ ràng; CORS cho phép origin của Vite dev server.
Verification	Kiểm tra độc lập Backend bằng Postman trước; sau đó kiểm tra FE ↔ BE bằng DevTools; cuối cùng kiểm chứng dữ liệu trong SQL Server.

1.1. Luồng hệ thống cần đạt

```
Browser (React @ http://localhost:5173)
  -> Axios HTTP request
  -> Spring Boot REST API @ http://localhost:8080/orchids
  -> Controller
  -> Service
  -> JpaRepository
  -> Hibernate/JDBC
  -> SQL Server
  <- JSON response
  <- React state update
  <- UI re-render
```

Nguyên tắc kiểm chứng: Không tích hợp mọi thứ cùng lúc. Luôn chứng minh Backend hoạt động độc lập trước bằng Postman, sau đó mới nối React. Khi lỗi, chẩn đoán từ boundary: Browser Network → CORS/URL → Controller → Service → Repository → SQL Server.

2. CÔNG NGHỆ, PHẦN MỀM VÀ ĐIỀU KIỆN CHUẨN BỊ

Thành phần	Baseline	Cách kiểm tra
JDK	JDK 21	java -version
Maven	Maven 3.6.3+	mvn -v
IDE Backend	IntelliJ IDEA	Mở project Maven không lỗi
Node.js / npm	Node LTS phù hợp Vite	node -v; npm -v
IDE Frontend	VS Code	Mở folder React và terminal
Database	Microsoft SQL Server	Kết nối được bằng SSMS/Azure Data Studio
API test	Postman	Gửi GET tới backend
Browser	Chrome/Edge	DevTools → Network/Console

2.1. Kiểm tra môi trường trước khi code

```
java -version
mvn -v
node -v
npm -v
```

CHECKPOINT PRE-01	Nội dung
Kết quả mong đợi	Bốn lệnh đều trả version; JDK là 21 hoặc baseline giảng viên chỉ định.
Cách kiểm chứng	Chụp/ghi lại output terminal.
Nếu chưa đạt	Nếu java/mvn/node/npm không nhận lệnh: kiểm tra PATH, JAVA_HOME hoặc cài đặt lại công cụ tương ứng.

3. CẤU TRÚC THƯ MỤC MỤC TIÊU

Để dễ debug, tách frontend và backend thành hai project ngang hàng trong một workspace:

```
SBA301-Lab05/
├── chapter15api/                # Spring Boot backend
│   ├── pom.xml
│   └── src/main/
│       ├── java/fu/se/chapter15api/
│       │   ├── Chapter15apiApplication.java
│       │   ├── configs/CorsConfig.java
│       │   ├── controllers/OrchidController.java
│       │   ├── pojos/Orchid.java
│       │   ├── repositories/IOrchidRepository.java
│       │   └── services/
│       │       ├── IOrchidService.java
│       │       └── OrchidService.java
│       └── resources/application.properties
└── chapter15reactjs/            # React/Vite frontend
    ├── .env
    ├── package.json
    └── src/
        ├── main.jsx
        ├── App.jsx
        ├── services/orchidApi.js
        └── components/
            ├── NavBar.jsx
            ├── ListOfOrchids.jsx
            └── EditOrchid.jsx
```

Lưu ý nguồn: Tên project chapter15api/chapter15reactjs được giữ theo Lab 05 gốc. Nếu lớp sử dụng convention khác, có thể đổi tên nhưng phải giữ package/import/URL nhất quán.

4. TÓM TẮT TOÀN BỘ CÁC BƯỚC THỰC HIỆN

#	Bước	Checkpoint chính
1	Chuẩn bị SQL Server và database	Kết nối DB thành công.
2	Tạo Spring Boot project	Project build được.
3	Thêm dependencies	Maven reload xanh.
4	Cấu hình datasource	Backend connect DB.
5	Tạo Entity	Hibernate tạo/nhận bảng.
6	Repository + Service	CRUD logic compile.
7	REST Controller	Postman GET/POST/PUT/DELETE đúng status.
8	CORS	Browser origin localhost:5173 được phép.
9	Tạo React/Vite project	npm run dev mở được trang.
10	Cấu hình .env và Axios	Request đi đúng http://localhost:8080/orchids.
11	List UI	GET data hiển thị trên React.
12	Edit/Create/Delete	CRUD từ UI cập nhật DB.
13	End-to-end verification	Network + API + DB cùng khớp.
14	Break/Fix	Sinh viên tự tạo và chẩn đoán ít nhất 1 lỗi tích hợp.

5. ACTIVITY 0 – CHUẨN BỊ SQL SERVER

Step 0.1 – Xác nhận SQL Server đang chạy

- Mở SQL Server Configuration Manager hoặc Services và xác nhận SQL Server instance đang Running.
- Mở SSMS/Azure Data Studio và đăng nhập bằng tài khoản phù hợp.

CHECKPOINT 0.1	Nội dung
Kết quả mong đợi	Có thể mở query window và chạy SELECT 1.
Cách kiểm chứng	Chạy SELECT 1 AS health_check; kết quả là 1.
Nếu chưa đạt	Kiểm tra instance name, authentication mode, firewall và TCP/IP.

Step 0.2 – Tạo database cho Lab 05

```
IF DB_ID('SBA301_OrchidDB') IS NULL
    CREATE DATABASE SBA301_OrchidDB;
GO
USE SBA301_OrchidDB;
GO
```

CHECKPOINT 0.2	Nội dung
Kết quả mong đợi	Database SBA301_OrchidDB xuất hiện.
Cách kiểm chứng	Refresh Databases trong SSMS hoặc chạy SELECT DB_NAME();
Nếu chưa đạt	Nếu thiếu quyền CREATE DATABASE, dùng database do giảng viên cấp và thay databaseName trong application.properties.

6. ACTIVITY 1 – TẠO SPRING BOOT REST BACKEND

Step 1.1 – Tạo Spring Initializr project

1. IntelliJ IDEA → File → New → Project → Spring Initializr.
2. Chọn Java, Maven, JDK 21, Packaging JAR.
3. Group: fu.se; Artifact/Name: chapter15api; package: fu.se.chapter15api.
4. Chọn dependencies: Spring Web, Spring Data JPA, MS SQL Server Driver.
5. Create và chờ IntelliJ tải dependencies.

CHECKPOINT 1.1	Nội dung
Kết quả mong đợi	Project mở được và pom.xml không báo dependency error.
Cách kiểm chứng	Maven tool window → Reload; chạy mvn -q -DskipTests compile.
Nếu chưa đạt	Nếu dependency không tải: kiểm tra Internet, Maven settings/proxy và JDK project SDK.

Step 1.2 – Kiểm tra dependencies trong pom.xml

```
<dependencies>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
        <groupId>com.microsoft.sqlserver</groupId>
        <artifactId>mssql-jdbc</artifactId>
        <scope>runtime</scope>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-test</artifactId>
        <scope>test</scope>
    </dependency>
</dependencies>
```

CHECKPOINT 1.2	Nội dung
Kết quả mong đợi	Maven resolve đủ Spring Web, JPA, JDBC SQL Server và test.
Cách kiểm chứng	Chạy mvn dependency:tree và tìm spring-boot-starter-web, spring-boot-starter-data-jpa, mssql-jdbc.
Nếu chưa đạt	Không tự ghi version rời nếu đang dùng Spring Boot parent/BOM trừ khi course baseline yêu cầu.

7. CẤU HÌNH KẾT NỐI SQL SERVER

Step 2.1 – application.properties

Lab gốc dùng SQL Server làm Data Source. Dưới đây là cấu hình course-ready tương đương; thay username/password/instance theo máy của sinh viên.

```
spring.application.name=chapter15api
server.port=8080

spring.datasource.url=jdbc:sqlserver://localhost:1433;databaseName=SBA301_OrchidDB;encrypt=true;trustServerCertificate=true
spring.datasource.username=sa
spring.datasource.password=YOUR_PASSWORD
spring.datasource.driver-class-name=com.microsoft.sqlserver.jdbc.SQLServerDriver

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
```

Không commit mật khẩu thật: Trong project cá nhân lab có thể dùng local password để học, nhưng trong repository dùng chung/production phải đưa secret ra environment/secret manager; không commit password thật vào Git.

Step 2.2 – Test kết nối DB trước khi tạo API

```
mvn spring-boot:run
```

- Quan sát log: application start không có Login failed / Cannot open database / TCP/IP error.
- Nếu ddl-auto=update, Hibernate có thể tạo bảng sau khi Entity xuất hiện. Ở thời điểm này chưa có Entity nên chỉ cần xác nhận DataSource không lỗi.

CHECKPOINT 2.2	Nội dung
Kết quả mong đợi	Backend start ở port 8080 và không lỗi datasource.
Cách kiểm chứng	Console có Started ...Application; không có stack trace connection failure.
Nếu chưa đạt	Kiểm tra URL, port 1433, databaseName, username/password, SQL Server TCP/IP.

8. ACTIVITY 2 – XÂY DỰNG PERSISTENCE VÀ BUSINESS LAYER

Step 3.1 – Tạo Orchid Entity

```
package fu.se.chapter15api.pojos;

import jakarta.persistence.*;

@Entity
@Table(name = "Orchid")
public class Orchid {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer orchidId;

    @Column(nullable = false)
    private String orchidName;

    private boolean isNatural;

    @Column(length = 1000)
    private String orchidDescription;

    private String orchidCategory;
    private boolean isAttractive;
    private String orchidURL;

    public Orchid() {}

    // Generate getters/setters for all fields in IntelliJ.
}
```

- Trong IntelliJ: Alt+Insert / Generate → Getter and Setter → chọn tất cả fields.
- Tên field giữ theo mô tả Lab: OrchidID, OrchidName, isNatural, orchidDescription, orchidCategory, isAttractive, orchidURL.

CHECKPOINT 3.1	Nội dung
Kết quả mong đợi	Project compile; khi backend chạy, bảng Orchid được tạo/cập nhật.
Cách kiểm chứng	SSMS: SELECT TABLE_NAME FROM INFORMATION_SCHEMA.TABLES WHERE TABLE_NAME = 'Orchid';
Nếu chưa đạt	Nếu Hibernate không tạo bảng: kiểm tra package nằm dưới package của @SpringBootApplication và ddl-auto.

Step 3.2 – Tạo Repository

```
package fu.se.chapter15api.repositories;

import fu.se.chapter15api.pojos.Orchid;
import org.springframework.data.jpa.repository.JpaRepository;

public interface IOrchidRepository extends JpaRepository<Orchid, Integer> {
}
```

CHECKPOINT 3.2	Nội dung
Kết quả mong đợi	Repository compile mà không cần class implementation.
Cách kiểm chứng	mvn -q -DskipTests compile
Nếu chưa đạt	Nếu Spring không scan repository: bảo đảm package repositories nằm dưới fu.se.chapter15api hoặc cấu hình scan rõ ràng.

Step 3.3 – Tạo Service interface

```
package fu.se.chapter15api.services;

import fu.se.chapter15api.pojos.Orchid;
import java.util.List;
import java.util.Optional;

public interface IOrchidService {
    List<Orchid> getAllOrchids();
    Optional<Orchid> getOrchidByID(int orchidID);
    Orchid insertOrchid(Orchid orchid);
```

```
    Orchid updateOrchid(int orchidID, Orchid orchid);
    void deleteOrchid(int orchidID);
}
```

Step 3.4 – Tạo Service implementation

```
package fu.se.chapter15api.services;

import fu.se.chapter15api.pojos.Orchid;
import fu.se.chapter15api.repositories.IOrchidRepository;
import org.springframework.stereotype.Service;
import java.util.List;
import java.util.Optional;

@Service
public class OrchidService implements IOrchidService {
    private final IOrchidRepository repository;

    public OrchidService(IOrchidRepository repository) {
        this.repository = repository;
    }

    @Override
    public List<Orchid> getAllOrchids() {
        return repository.findAll();
    }

    @Override
    public Optional<Orchid> getOrchidByID(int orchidID) {
        return repository.findById(orchidID);
    }

    @Override
    public Orchid insertOrchid(Orchid orchid) {
        orchid.setOrchidId(null);
        return repository.save(orchid);
    }

    @Override
    public Orchid updateOrchid(int orchidID, Orchid input) {
        Orchid current = repository.findById(orchidID)
                .orElseThrow(() -> new IllegalArgumentException("Orchid not found: " + orchidID));
        current.setOrchidName(input.getOrchidName());
        current.setNatural(input.isNatural());
        current.setOrchidDescription(input.getOrchidDescription());
        current.setOrchidCategory(input.getOrchidCategory());
        current.setAttractive(input.isAttractive());
        current.setOrchidURL(input.getOrchidURL());
        return repository.save(current);
    }

    @Override
    public void deleteOrchid(int orchidID) {
        repository.deleteById(orchidID);
    }
}
```

Ghi chú course-ready: File Lab gốc minh họa service đơn giản. Bản hướng dẫn dùng constructor injection và cập nhật đầy đủ field để sinh viên nhìn rõ boundary Service → Repository. Có thể dùng tên getter/setter khác tùy cách IDE sinh method cho boolean field.

CHECKPOINT 3.4	Nội dung
Kết quả mong đợi	Service compile và Spring có thể inject repository.
Cách kiểm chứng	mvn -q -DskipTests compile
Nếu chưa đạt	Nếu compile lỗi boolean getter/setter: xem tên method thật mà IDE sinh, ví dụ isNatural()/setNatural(...) hoặc getIsNatural()/setIsNatural(...).

9. ACTIVITY 2 – TẠO REST CONTROLLER VÀ CRUD ENDPOINTS

```
package fu.se.chapter15api.controllers;

import fu.se.chapter15api.pojos.Orchid;
import fu.se.chapter15api.services.IOrchidService;
import org.springframework.http.*;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/orchids")
public class OrchidController {
    private final IOrchidService service;

    public OrchidController(IOrchidService service) {
        this.service = service;
    }

    @GetMapping("/")
    public ResponseEntity<List<Orchid>> fetchAll() {
        return ResponseEntity.ok(service.getAllOrchids());
    }

    @GetMapping("/{id}")
    public ResponseEntity<Orchid> getById(@PathVariable int id) {
        return service.getOrchidByID(id)
                .map(ResponseEntity::ok)
                .orElseGet(() -> ResponseEntity.notFound().build());
    }

    @PostMapping("/")
    public ResponseEntity<Orchid> saveOrchid(@RequestBody Orchid orchid) {
        Orchid created = service.insertOrchid(orchid);
        return ResponseEntity.status(HttpStatus.CREATED).body(created);
    }

    @PutMapping("/{id}")
    public ResponseEntity<Orchid> updateOrchid(@PathVariable int id, @RequestBody Orchid orchid) {
        try {
            return ResponseEntity.ok(service.updateOrchid(id, orchid));
        } catch (IllegalArgumentException ex) {
            return ResponseEntity.notFound().build();
        }
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteOrchid(@PathVariable int id) {
        if (service.getOrchidByID(id).isEmpty()) {
            return ResponseEntity.notFound().build();
        }
        service.deleteOrchid(id);
        return ResponseEntity.noContent().build();
    }
}
```

Step 4.1 – Chạy backend

```
mvn spring-boot:run
```

CHECKPOINT 4.1	Nội dung
Kết quả mong đợi	Backend listening ở http://localhost:8080.
Cách kiểm chứng	Browser/Postman: GET http://localhost:8080/orchids/ trả 200 và JSON [] hoặc danh sách.
Nếu chưa đạt	Nếu 404: kiểm tra @RequestMapping và dấu /. Nếu 500: xem stack trace root cause.

10. KIỂM THỬ BACKEND BẰNG POSTMAN TRƯỚC KHI TÍCH HỢP REACT

Test	Method	URL	Expected
T01	GET	http://localhost:8080/orchids/	200 OK + JSON list
T02	POST	http://localhost:8080/orchids/	201 Created + object có id
T03	GET	http://localhost:8080/orchids/{id}	200 hoặc 404

T04	PUT	http://localhost:8080/orchids/{id}	200 hoặc 404
T05	DELETE	http://localhost:8080/orchids/{id}	204 hoặc 404

Step 5.1 – POST seed data

```
{
  "orchidName": "Taichung Beauty",
  "natural": true,
  "orchidDescription": "Demo orchid for SBA301 Lab 05",
  "orchidCategory": "Cattleya",
  "attractive": true,
  "orchidURL": "https://example.com/orchid.jpg"
}
```

- Postman → POST → Body → raw → JSON → dán payload → Send.

CHECKPOINT 5.1	Nội dung
Kết quả mong đợi	HTTP 201; response có orchidId khác null.
Cách kiểm chứng	SSMS: SELECT * FROM Orchid ORDER BY orchidId DESC;
Nếu chưa đạt	400/415: kiểm tra Content-Type application/json và JSON field names; 500: xem SQL/JPA log.

Step 5.2 – PUT/DELETE và hậu điều kiện

- PUT /orchids/{id}: thay orchidName hoặc category; Send; sau đó GET lại đúng id.
- DELETE /orchids/{id}: nhận 204; sau đó GET cùng id phải 404.

CHECKPOINT 5.2	Nội dung
Kết quả mong đợi	API contract và database cùng phản ánh thay đổi.
Cách kiểm chứng	So sánh response Postman với SELECT trong SQL Server.
Nếu chưa đạt	Nếu API trả success nhưng DB không đổi: kiểm tra repository.save(), transaction/error log và đúng databaseName.

11. ACTIVITY 2 – CẤU HÌNH CORS CHO REACT ↔ SPRING BOOT

Lab gốc nhấn mạnh CORS vì frontend chạy ở origin khác backend. Với Vite mặc định, React thường chạy ở http://localhost:5173 còn Spring Boot chạy ở http://localhost:8080; khác port nghĩa là khác origin.

Step 6.1 – Tạo CorsConfig.java

```
package fu.se.chapter15api.configs;

import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.CorsRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

@Configuration
public class CorsConfig implements WebMvcConfigurer {
    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/**")
                .allowedOrigins("http://localhost:5173")
                .allowedMethods("GET", "POST", "PUT", "DELETE", "OPTIONS")
                .allowedHeaders("*");
    }
}
```

Tại sao không dùng * mặc định? Trong lab development, exact origin giúp sinh viên hiểu đúng security boundary. Nếu sau này gửi credential/cookie, wildcard origin không phù hợp; production phải cấu hình theo domain thực tế.

Step 6.2 – Restart backend và kiểm tra CORS

- Sau khi thêm config, stop và start lại Spring Boot.
- CORS không thể kiểm chứng đầy đủ bằng Postman vì browser mới áp Same-Origin Policy. Kiểm tra thật sau khi React gửi request.

CHECKPOINT 6.2	Nội dung
Kết quả mong đợi	Backend restart không lỗi bean/config.
Cách kiểm chứng	Console Started ...Application.
Nếu chưa đạt	Nếu tạo nhiều CORS config trùng nhau, tạm giữ một cách duy nhất để giảm nhiễu khi học.

12. ACTIVITY 3 – TẠO FRONTEND REACTJS BẰNG VITE

Step 7.1 – Scaffold project

```
cd SBA301-Lab05
npm create vite@latest chapter15reactjs
# Chọn: React
# Chọn: JavaScript
cd chapter15reactjs
npm install
```

CHECKPOINT 7.1	Nội dung
Kết quả mong đợi	node_modules được cài và package.json tồn tại.
Cách kiểm chứng	npm run dev; terminal hiển thị Local URL.
Nếu chưa đạt	Nếu npm lỗi permission/cache: kiểm tra Node/npm version, terminal quyền phù hợp, xóa node_modules/package-lock nếu cần rồi npm install lại.

Step 7.2 – Cài thư viện dùng trong Lab

```
npm install axios react-router-dom react-bootstrap bootstrap
```

- Axios: HTTP client.
- react-router-dom: điều hướng SPA.
- react-bootstrap + bootstrap: UI nhanh cho lab; không bắt buộc về kiến trúc nhưng phù hợp tài liệu trước.

CHECKPOINT 7.2	Nội dung
Kết quả mong đợi	package.json có dependencies mới.

Cách kiểm chứng	npm list axios react-router-dom react-bootstrap bootstrap
Nếu chưa đạt	Nếu npm báo peer dependency: ghi nhận version, nhưng không nâng/hạ ngẫu nhiên nếu lớp đã khóa baseline.

13. CẤU HÌNH API URL VÀ AXIOS CLIENT

Step 8.1 – Tạo file .env ở root frontend

```
VITE_API_URL=http://localhost:8080/orchids
```

Quy tắc Vite: Biến cần đọc ở client dùng prefix VITE_. Sau khi sửa .env phải restart npm run dev. Không đặt password/database secret trong VITE_* vì giá trị client-side có thể được đưa vào bundle.

Step 8.2 – Tạo src/services/orchidApi.js

```
import axios from "axios";

const API_URL = import.meta.env.VITE_API_URL;

const api = axios.create({
  baseURL: API_URL,
  headers: { "Content-Type": "application/json" },
});

export const getOrchids = () => api.get("/");
export const getOrchid = (id) => api.get(`/${id}`);
export const createOrchid = (payload) => api.post("/", payload);
export const updateOrchid = (id, payload) => api.put(`/${id}`, payload);
export const deleteOrchid = (id) => api.delete(`/${id}`);

export default api;
```

CHECKPOINT 8.2	Nội dung
Kết quả mong đợi	Frontend compile không báo import/env error.
Cách kiểm chứng	npm run dev; mở Console, không có ReferenceError import.meta.env.
Nếu chưa đạt	Nếu baseURL undefined: file phải tên .env, đặt ở root cạnh package.json, và restart Vite.

14. TẠO ROOT COMPONENT VÀ ROUTING

Step 9.1 – main.jsx

```
import React from "react";
import ReactDOM from "react-dom/client";
import { BrowserRouter } from "react-router-dom";
import "bootstrap/dist/css/bootstrap.min.css";
import App from "./App.jsx";

ReactDOM.createRoot(document.getElementById("root")).render(
  <React.StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </React.StrictMode>
);
```

Step 9.2 – App.jsx

```
import { Routes, Route } from "react-router-dom";
import NavBar from "./components/NavBar";
import ListOfOrchids from "./components/ListOfOrchids";
import EditOrchid from "./components/EditOrchid";

export default function App() {
  return (
    <>
      <NavBar />
      <div className="container py-4">
        <Routes>
```

```
          <Route path="/" element={<ListOfOrchids />} />
          <Route path="/orchids/:id/edit" element={<EditOrchid />} />
          <Route path="/orchids/new" element={<EditOrchid createMode />} />
        </Routes>
      </div>
    </>
  );
}
```

Step 9.3 – NavBar.jsx

```
import { Navbar, Container, Nav } from "react-bootstrap";
import { Link } from "react-router-dom";

export default function NavBar() {
  return (
    <Navbar bg="dark" data-bs-theme="dark">
      <Container>
        <Navbar.Brand as={Link} to="/">Orchid Management</Navbar.Brand>
        <Nav className="me-auto">
          <Nav.Link as={Link} to="/">List</Nav.Link>
          <Nav.Link as={Link} to="/orchids/new">Create</Nav.Link>
        </Nav>
      </Container>
    </Navbar>
  );
}
```

CHECKPOINT 9	Nội dung
Kết quả mong đợi	Trang React mở được; Navbar và routes không lỗi.
Cách kiểm chứng	Browser http://localhost:5173; click List/Create không full-page reload.
Nếu chưa đạt	Nếu trắng trang: kiểm tra Console; thường do import path/export mismatch.

15. HIỂN THỊ DANH SÁCH ORCHID TỪ BACKEND

```
import { useEffect, useState } from "react";
import { Link } from "react-router-dom";
import { getOrchids, deleteOrchid } from "../services/orchidApi";

export default function ListOfOrchids() {
  const [orchids, setOrchids] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState("");

  const loadData = async () => {
    try {
      setLoading(true);
      const response = await getOrchids();
      setOrchids(response.data);
      setError("");
    } catch (e) {
      setError(e.message);
    } finally {
      setLoading(false);
    }
  };

  useEffect(() => { loadData(); }, []);

  const handleDelete = async (id) => {
    if (!window.confirm("Delete this orchid?")) return;
    try {
      await deleteOrchid(id);
      await loadData();
    } catch (e) {
      setError(e.message);
    }
  };

  if (loading) return <p>Loading...</p>;
  if (error) return <div className="alert alert-danger">{error}</div>;
```

```
  return (
    <div>
      <div className="d-flex justify-content-between align-items-center mb-3">
        <h2>Orchids</h2>
        <Link className="btn btn-primary" to="/orchids/new">Create Orchid</Link>
      </div>
      <table className="table table-striped">
        <thead><tr><th>ID</th><th>Name</th><th>Category</th><th>Natural</th><th>Actions</th></tr></thead>
        <tbody>
          {orchids.map(o => (
            <tr key={o.orchidId}>
              <td>{o.orchidId}</td>
              <td>{o.orchidName}</td>
              <td>{o.orchidCategory}</td>
              <td>{o.natural ? "Yes" : "No"}</td>
              <td>
                <Link className="btn btn-sm btn-warning me-2" to={`/orchids/${o.orchidId}/edit`}>Edit</Link>
                <button className="btn btn-sm btn-danger" onClick={() => handleDelete(o.orchidId)}>Delete</button>
              </td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  );
}
```

Step 10.1 – Chạy và kiểm chứng GET integration

```
# Terminal 1 - backend
cd chapter15api
mvn spring-boot:run

# Terminal 2 - frontend
cd chapter15reactjs
npm run dev
```

- Mở http://localhost:5173.
- DevTools → Network → lọc Fetch/XHR → reload trang.
- Chọn request /orchids/ và kiểm tra Request URL, Status 200, Response JSON.

CHECKPOINT 10.1	Nội dung
Kết quả mong đợi	Danh sách từ SQL Server xuất hiện trên React. Network có GET 200.
Cách kiểm chứng	Đối chiếu 3 nơi: UI count = response array length = số row SELECT * FROM Orchid.
Nếu chưa đạt	Nếu Postman OK nhưng browser fail: ưu tiên kiểm tra CORS hoặc API URL, không sửa Repository trước.

16. CREATE / EDIT ORCHID TỪ REACT

```
import { useEffect, useState } from "react";
import { useNavigate, useParams } from "react-router-dom";
import { createOrchid, getOrchid, updateOrchid } from "../services/orchidApi";

const emptyForm = {
  orchidName: "", natural: false, orchidDescription: "",
  orchidCategory: "", attractive: false, orchidURL: ""
};

export default function EditOrchid({ createMode = false }) {
  const { id } = useParams();
  const navigate = useNavigate();
  const [form, setForm] = useState(emptyForm);
  const [error, setError] = useState("");

  useEffect(() => {
    if (!createMode && id) {
      getOrchid(id).then(r => setForm(r.data)).catch(e => setError(e.message));
    }
  }, [createMode, id]);

  const change = (e) => {
    const { name, value, type, checked } = e.target;
    setForm(f => ({ ...f, [name]: type === "checkbox" ? checked : value }));
```

```
  };

  const submit = async (e) => {
    e.preventDefault();
    try {
      if (createMode) await createOrchid(form);
      else await updateOrchid(id, form);
      navigate("/");
    } catch (e) { setError(e.message); }
  };

  return (
    <form onSubmit={submit}>
      <h2>{createMode ? "Create Orchid" : "Edit Orchid"}</h2>
      {error && <div className="alert alert-danger">{error}</div>}
      <label className="form-label">Name</label>
      <input className="form-control mb-2" name="orchidName" value={form.orchidName ?? ""} onChange={change}
required />
      <label className="form-label">Category</label>
      <input className="form-control mb-2" name="orchidCategory" value={form.orchidCategory ?? ""}
onChange={change} />
      <label className="form-label">Description</label>
      <textarea className="form-control mb-2" name="orchidDescription" value={form.orchidDescription ?? ""}
onChange={change} />
      <label className="form-label">Image URL</label>
      <input className="form-control mb-2" name="orchidURL" value={form.orchidURL ?? ""} onChange={change} />
      <div className="form-check mb-2"><input className="form-check-input" type="checkbox" name="natural"
checked={!!form.natural} onChange={change} /><label className="form-check-label">Natural</label></div>
      <div className="form-check mb-3"><input className="form-check-input" type="checkbox" name="attractive"
checked={!!form.attractive} onChange={change} /><label className="form-check-label">Attractive</label></div>
      <button className="btn btn-success" type="submit">Save</button>
    </form>
  );
}
```

Step 11.1 – Kiểm chứng Create

6. Click Create → nhập Name/Category → Save.
7. DevTools Network: POST /orchids/ phải 201.
8. UI quay về list và hiển thị row mới.
9. SSMS SELECT: row mới tồn tại, ID sinh tự động.

CHECKPOINT 11.1	Nội dung
Kết quả mong đợi	Một thao tác Create được chứng minh ở Network, UI và DB.
Cách kiểm chứng	Lưu bằng chứng request/response + SELECT row mới.
Nếu chưa đạt	Nếu POST 201 nhưng list chưa cập nhật: kiểm tra navigation/reload; nếu POST fail, mở response body/console.

Step 11.2 – Kiểm chứng Update

10. Click Edit trên một row → component GET detail.
11. Sửa Name → Save → PUT /orchids/{id}.
12. Network status 200; GET/list sau đó hiển thị tên mới; SQL Server cũng cập nhật.

CHECKPOINT 11.2	Nội dung
Kết quả mong đợi	PUT tác động đúng 1 record và giữ nguyên ID.
Cách kiểm chứng	So sánh before/after UI + DB.
Nếu chưa đạt	Nếu field boolean không cập nhật: kiểm tra JSON property name và getter/setter backend.

Step 11.3 – Kiểm chứng Delete

13. Click Delete → Confirm.
14. Network DELETE /orchids/{id} → 204 No Content.
15. GET lại id phải 404; row biến mất khỏi DB.

CHECKPOINT 11.3	Nội dung
Kết quả mong đợi	Delete có hậu điều kiện rõ ràng.
Cách kiểm chứng	Network 204 + SELECT không còn row + GET detail 404.
Nếu chưa đạt	Nếu UI xóa nhưng DB còn row, xác minh UI có đang chỉ filter local hay thật sự gọi DELETE.

17. TRACE END-TO-END MỘT REQUEST

Sinh viên phải có khả năng giải thích một request không chỉ "chạy được" mà còn đi qua các lớp nào.

```
React ListOfOrchids.loadData()
  -> getOrchids() in orchidApi.js
  -> Axios GET http://localhost:8080/orchids/
  -> Browser CORS check
  -> OrchidController.fetchAll()
  -> OrchidService.getAllOrchids()
  -> IOrchidRepository.findAll()
  -> Hibernate/JDBC SELECT
  -> SQL Server
  <- rows
  <- List<Orchid>
  <- JSON 200
  <- Axios response.data
  -> setOrchids(...)
  -> React re-render table
```

Điểm quan sát	Bằng chứng
Browser Network	Request URL, method, status, response JSON, CORS headers.
Spring console	Mapping/exception/SQL log.
IDE breakpoint	Controller → Service chain.
SQL Server	Row state trước/sau thao tác.
React UI	State được render thành table/form.

18. TEST MATRIX TỔNG HỢP

ID	Kịch bản	API kỳ vọng	UI kỳ vọng	DB kỳ vọng
E01	Mở list	GET 200	Hiển thị rows	Không đổi
E02	Create valid	POST 201	Row mới	INSERT row
E03	Edit existing	GET 200 + PUT 200	Giá trị mới	UPDATE row
E04	Delete existing	DELETE 204	Row biến mất	DELETE row
E05	GET missing	404	Thông báo/không crash	Không đổi
E06	Backend stop	Network error	Error alert	Không đổi
E07	Sai API URL	Request fail	Error alert	Không đổi
E08	CORS bị chặn	Browser CORS error	Không load data	Backend có thể vẫn chạy

19. DEBUGGING – LỖI THƯỜNG GẶP VÀ CÁCH XỬ LÝ

Triệu chứng	Khả năng	Cách xác minh/xử lý
React trắng trang	JS/import/runtime error	DevTools Console; kiểm tra import/export và JSX.
ERR_CONNECTION_REFUSED	Backend chưa chạy / sai port	Mở http://localhost:8080/orchids/ trực tiếp hoặc Postman.
CORS policy error	Origin chưa được allow	Kiểm tra http://localhost:5173 chính xác; restart backend.
404	Sai path hoặc dấu /	So request URL với @RequestMapping/@GetMapping.
405	Sai HTTP method	Network method vs controller mapping.
415	Sai Content-Type	Axios/Postman gửi application/json.
400	JSON/binding lỗi	Xem response + field names/types.
500	Application/DB error	Đọc root cause stack trace; kiểm tra SQL log.
Login failed SQL Server	Credentials sai	Test login bằng SSMS và sửa datasource.
Could not open connection	Port/TCP/instance	SQL Server TCP/IP, port 1433, firewall.
Response OK nhưng UI không đổi	State/navigation/loadData	Xem Network response rồi kiểm tra setState/reload.
Boolean hiển thị sai	Property/getter naming	Inspect JSON actual property names, đồng bộ frontend/backend.

19.1. Debugging decision tree

```
UI lỗi
 |- Browser có request không?
 |   |- Không -> React event/router/api function
```

```
|   `- Có -> status?
|       |- Network/CORS -> URL/origin/CORS/backend running
|       |- 404/405 -> route/method
|       |- 400/415 -> JSON/Content-Type/binding
|       `- 500 -> Spring stack trace -> Service/Repository/DB
`- Request 2xx nhưng dữ liệu sai
    -> inspect response JSON
    -> inspect React state/render
    -> inspect SQL Server row state
```

20. BREAK-IT → DIAGNOSE → FIX WORKSHOP

Break	Triệu chứng dự kiến	Evidence phải nộp
Đổi VITE_API_URL sang port 9999	ERR_CONNECTION_REFUSED	Network screenshot + fix note.
Xóa origin 5173 khỏi CORS	CORS policy blocked	Console + response headers + fix.
Đổi GET thành POST ở api client	405 Method Not Allowed	Network request method + controller mapping.
Gửi text/plain cho POST	415 hoặc binding fail	Headers + status + corrected JSON request.
Đổi databaseName sai	Backend startup/DB error	Stack trace root cause + corrected property.
Dừng SQL Server khi backend chạy	Persistence request fail	Spring/JDBC error + recovery evidence.

21. GHI CHÚ THỰC TIỄN / PRODUCTION DIRECTION

Course baseline vs production: Lab này cố ý đơn giản hóa để học tích hợp React ↔ Spring Boot. Trong dự án production, không mặc định expose JPA Entity trực tiếp; thường dùng request/response DTO, validation, global error contract, authentication/authorization, API versioning khi cần, observability, automated tests và secret management.

Lab-level	Production direction
VITE_API_URL localhost	Environment-specific config / reverse proxy / deployment domain.
Entity làm request/response	DTO + validation + mapping.
CORS allow localhost	Allowlist domain thật; review credentials/security policy.
Manual Postman test	Automated controller/service/integration tests.
Password trong local properties	Environment variable / secret store.
Simple CRUD	Pagination/search, concurrency, audit, error contract.

22. CHECKLIST HOÀN THÀNH LAB 05

- ☐ Backend Maven build thành công.
- ☐ SQL Server kết nối thành công và có bảng Orchid.
- ☐ GET /orchids/ trả 200.
- ☐ POST trả 201 và row mới xuất hiện trong DB.
- ☐ PUT cập nhật đúng row.
- ☐ DELETE trả 204 và GET lại trả 404.
- ☐ CORS cho phép frontend origin.
- ☐ React chạy bằng Vite tại localhost:5173.
- ☐ .env chứa VITE_API_URL đúng.
- ☐ Axios client dùng base URL tập trung.
- ☐ ListOfOrchids hiển thị dữ liệu backend.
- ☐ Create/Edit/Delete từ UI hoạt động.
- ☐ DevTools Network chứng minh request/response.
- ☐ Có test matrix positive + negative.
- ☐ Có ít nhất 1 Break-It → Diagnose → Fix note.

23. BỘ BẰNG CHỨNG KHUYẾN NGHỊ SINH VIÊN NỘP

Evidence	Nội dung
E1 – Build	Terminal mvn compile / application started.
E2 – DB	SSMS SELECT dữ liệu Orchid.

E3 – Postman	GET + POST + PUT + DELETE với status.
E4 – CORS/Network	Browser Network cho request từ 5173 → 8080.
E5 – UI	List + Create/Edit form.
E6 – Trace	Một request flow React → DB → React.
E7 – Break/Fix	Một lỗi do sinh viên cố ý tạo và cách chẩn đoán.
E8 – README	Cách chạy BE/FE, ports, DB, API URL, known issues.

24. QUICK RUN CARD – CHẠY LẠI PROJECT SAU KHI ĐÃ HOÀN THÀNH

```
1) Start SQL Server

2) Backend
   cd chapter15api
   mvn spring-boot:run
   Verify: GET http://localhost:8080/orchids/

3) Frontend
   cd chapter15reactjs
   npm install          # chỉ cần khi clone/cài mới
   npm run dev
   Open: http://localhost:5173

4) Verify
   Browser DevTools -> Network
   SQL Server -> SELECT * FROM Orchid;
```